

Lab 3 — Quantization: Dynamic PTQ, Static PTQ, and ONNX

Course: Big Data at the Edge — Week 3 **Weight:** 10% of final grade

Overview

Students apply post-training quantization to the same CIFAR-10-adapted MobileNetV2 used in Lab 2, comparing:

1. **Dynamic PTQ** (`torch.ao.quantization.quantize_dynamic`) — weights quantized ahead of time, activations quantized on the fly.
2. **Static PTQ** via **FX Graph Mode quantization** (`torch.ao.quantization.quantize_fx`) — weights and activations both quantized ahead of time, using a small calibration set. FX mode is used specifically because it auto-fuses Conv+BN+ReLU patterns, which would be tedious to do by hand across MobileNetV2's inverted residual blocks.
3. **ONNX conversion** — exporting the FP32 model to ONNX and then quantizing the ONNX graph directly via `onnxruntime.quantization.quantize_dynamic`, discussed as the more portable alternative to exporting an already-quantized PyTorch model.

All quantized-model measurements are taken on **CPU**, since PyTorch's standard int8 quantized kernels don't run on CUDA.

What runs where (GPU vs. CPU)

Section	Device
1. FP32 baseline (train/load)	GPU if available (falls back to CPU automatically)
2. Dynamic PTQ	CPU only
3. Static PTQ (FX graph mode) — prepare, calibrate, convert, and the resulting quantized model	CPU only
4. Summary table	N/A — just aggregates the numbers above
5. ONNX export of the FP32 model	Same device as the FP32 model (CPU, per above)
5. ONNX Runtime quantization + verifying the quantized ONNX model	CPU only

This isn't just a performance suggestion — `torch.ao.quantization`'s int8 kernels will raise a `RuntimeError` if you try to run them on a CUDA tensor, and ONNX Runtime's post-training quantization is a graph-level transformation with no GPU involved either way. Explicitly move models to `cpu_device` (`torch.device("cpu")`) before quantizing, evaluating, or benchmarking them from Section 2 onward — in both the notebook and the script, `cpu_device` is already defined for you in Section 0/Setup. This holds even when running on DAS-5 with a GPU allocated: the GPU there only helps Section 1 train faster; everything from Section 2 onward still runs on CPU on that same node.

Learning objectives

- Implement dynamic and static post-training quantization in PyTorch.
- Explain why dynamic PTQ has limited impact on convolution-heavy architectures like MobileNetV2.
- Use FX graph mode quantization (prepare/calibrate/convert) correctly, including selecting a quantization backend (`fbgemm` vs. `qnnpack`).
- Quantize an ONNX graph directly with ONNX Runtime, and articulate why this is often more portable across edge runtimes than exporting an already-quantized framework-specific model.
- Connect INT8 quantization back to floating-point representation fundamentals from lecture.

Files

File	Purpose
lab3_quantization.ipynb	Student notebook — fill in the # TODO cells. Use this if you're working locally/interactively.
lab3_quantization.py	Same assignment as a plain script — fill in the same # TODOs. Use this on DAS-5, since notebooks can't be submitted as SLURM batch jobs.
submit_lab3_quantization.sbatch	SLURM submission script for running lab3_quantization.py as a batch job on DAS-5.
utils.py	Data loading, model, train/eval loop, size/latency measurement, dynamic quantization wrapper, and a calibration helper. The FX prepare/calibrate/convert flow and the ONNX Runtime quantization live in the notebook/script itself (core lab content).
requirements.txt	Python dependencies.

Notebook vs. script vs. sbatch

- **Notebook** (lab3_quantization.ipynb) — interactive exploration, learning, and debugging. Convenient for iterating cell-by-cell, but Jupyter notebooks can't be submitted directly as SLURM batch jobs.
- **Script** (lab3_quantization.py) — the same experiment, restructured to run non-interactively from the command line. It mirrors the notebook section-by-section (same # TODOs, same variable names), so work you did in one carries over directly to the other.
- **sbatch** (submit_lab3_quantization.sbatch) — requests cluster resources (GPU, CPUs, memory, time) from SLURM and runs the script on a compute node. This is what you actually submit on DAS-5.

You're not submitting the notebook to SLURM because Slurm runs plain executables/scripts on compute nodes — there's no Jupyter kernel or display server there to run a .ipynb against.

Setup (local)

```
python3 -m venv .venv
source .venv/bin/activate    # Windows: .venv\Scripts\activate
pip install -r requirements.txt
jupyter notebook lab3_quantization.ipynb
```

If you already have a fine-tuned checkpoint from Lab 2, you may load it directly (utils.load_checkpoint in the notebook, or --checkpoint in the script — see below) instead of retraining — check the assignment instructions or ask your instructor whether that's expected for your section.

Running on DAS-5 (SLURM batch job)

Fill in the # TODOs in lab3_quantization.py (the script mirrors the notebook section-by-section, so anything you worked out interactively carries over directly).

Set up the environment with create_env.sh (provided in Lab-1 in envs.zip) rather than installing into your home directory — DAS-5 home directories have limited quota, and this installs everything under /var/scratch/\$USER instead:

```
cd envs
bash create_env.sh
```

This only needs to be run once — skip it if you already set up your environment for Lab 1 or Lab 2. Then submit the job:

```
cd Lab3_Quantization
mkdir -p logs
sbatch submit_lab3_quantization.sbatch
```

Create the `logs/` directory *before* submitting — SLURM needs it to already exist to write `logs/%x_%j.out`, since that happens before the job script itself runs. `submit_lab3_quantization.sbatch` writes CIFAR-10, checkpoints, the ONNX exports, and the summary table under `/var/scratch/$USER` as well, and pins `--constraint=GTXT1080Ti` to request a specific GPU type — run `sinfo -o "%N %f"` to see other node features available if you want a different one.

The GPU is used only to train the FP32 baseline faster in Section 1 — every quantized model (Sections 2–5) is measured on **CPU**, exactly as in the notebook, since PyTorch’s standard int8 quantized kernels don’t run on CUDA. If you already have a fine-tuned checkpoint from Lab 2, add `--checkpoint /path/to/your/checkpoint.pt` to the `srunch python lab3_quantization.py ...` line in the sbatch file to skip training and load it directly instead.

If you run the batch version, the deliverables are the same as listed above — just submit `lab3_quantization.py` (completed) together with the job’s `.out` log; your output directory will also contain `summary.csv` and the exported `mobilenet_v2_fp32.onnx / mobilenet_v2_int8.onnx` files as supporting evidence.

Checking your job

```
squeue -u $USER
```

shows whether your job is pending, running, or finished. Once it’s done (or to check resource usage for a completed job):

```
sacct -j <JOB_ID>
```

If the job fails

Check `logs/<job-name>_<job-id>.out` first — stdout and stderr both land there. Common things to check:

- The `myenv` conda environment wasn’t created yet, or `requirements_das5.txt` wasn’t fully installed — re-run `create_env.sh`.
- `--constraint=GTXT1080Ti` requested a GPU type that’s unavailable or busy — check `sinfo -o "%N %f"` for other options.
- The job ran out of time (`--time` in the sbatch file) — this is more likely if you increased `--baseline-epochs` or kept `--train-subset-fraction 1.0` combined with a slow queue.
- `--calibration-batches` is too small or unrepresentative for Section 3 — if static PTQ accuracy looks unexpectedly bad, that’s usually a calibration issue rather than a script bug.

Task summary

1. Train/load an FP32 baseline; measure accuracy, size, CPU latency.
2. Dynamic PTQ on the classifier **Linear** layer; measure and reflect on why the effect is small for this architecture.
3. Static PTQ via FX graph mode (prepare → calibrate → convert); measure and compare.
4. Build a summary table across FP32 / dynamic / static.
5. Export the FP32 model to ONNX, quantize the ONNX graph with `onnxruntime.quantization`, and verify outputs against the FP32 ONNX graph.

Deliverables

- This completed notebook (or the completed script, `lab3_quantization.py`, if you ran the batch version on DAS-5).
- A table comparing FP32 baseline vs. dynamic PTQ vs. static PTQ across size / accuracy / latency.
- A quantized ONNX model (via the ONNX Runtime quantization route), verified against the FP32 ONNX baseline.
- Written answers to the reflection questions in each section.

Grading rubric (10%)

Component	Weight
Dynamic PTQ correctly implemented and measured	15%
Static PTQ (FX graph mode) correctly implemented, calibrated, and measured	30%
Summary comparison table	10%
ONNX export + ONNX Runtime quantization + numerical verification	20%
Reflection questions — depth of reasoning, not just correctness	25%

Tips

- We use FX graph mode quantization (`torch.ao.quantization.quantize_fx`) instead of the older eager-mode API, specifically so you don't have to hand-list MobileNetV2's nested `Conv2dNormActivation` fusion groups before quantizing — FX mode fuses Conv+BN+ReLU patterns for you automatically.
- `torch.backends.quantized.engine` is set to `"fbgemm"` here, which targets x86 CPUs. If you were deploying to an ARM-based edge device instead, you'd use `"qnnpack"` — worth remembering, since which backend you pick depends entirely on your target hardware.
- Section 5's Option A (exporting an already-quantized PyTorch model directly to ONNX) is exploratory — PyTorch-quantized-model → ONNX export support has been inconsistent across versions, so don't worry if it fails for you. Make sure you complete Option B (ONNX Runtime quantization) correctly, since that's the part that's graded.
- If your static PTQ accuracy drops more than expected, check that your calibration batches (`n_batches` in `utils.calibrate`) actually cover a reasonable mix of classes — a calibration set that's too small or unrepresentative will give the observers a skewed picture of activation ranges.
- Getting a `RuntimeError` mentioning quantized engines or unsupported device during Sections 2–5? Double-check you moved the model to `cpu_device` first — see “What runs where (GPU vs. CPU)” above.